

異常

An example: 学生成绩导入器

- 读取文件
- 解析分数
- 保存结果
- 提示用户

如果不用异常： 错误码会污染正常逻辑

```
int result = importScores("scores.csv");
```

```
if (result == 0) {  
    System.out.println("导入成功");  
} else if (result == 1) {  
    System.out.println("文件不存在");  
} else if (result == 2) {  
    System.out.println("分数格式错误");  
} else if (result == 3) {  
    System.out.println("保存失败");  
}
```

问题

- 返回值既要表达正常结果，又要表达错误原因
- 调用者必须记住每个数字代表什么
- 错误容易被忽略：有人可能只检查 `result == 0`
- 业务代码和错误处理混在一起

异常的基本思想

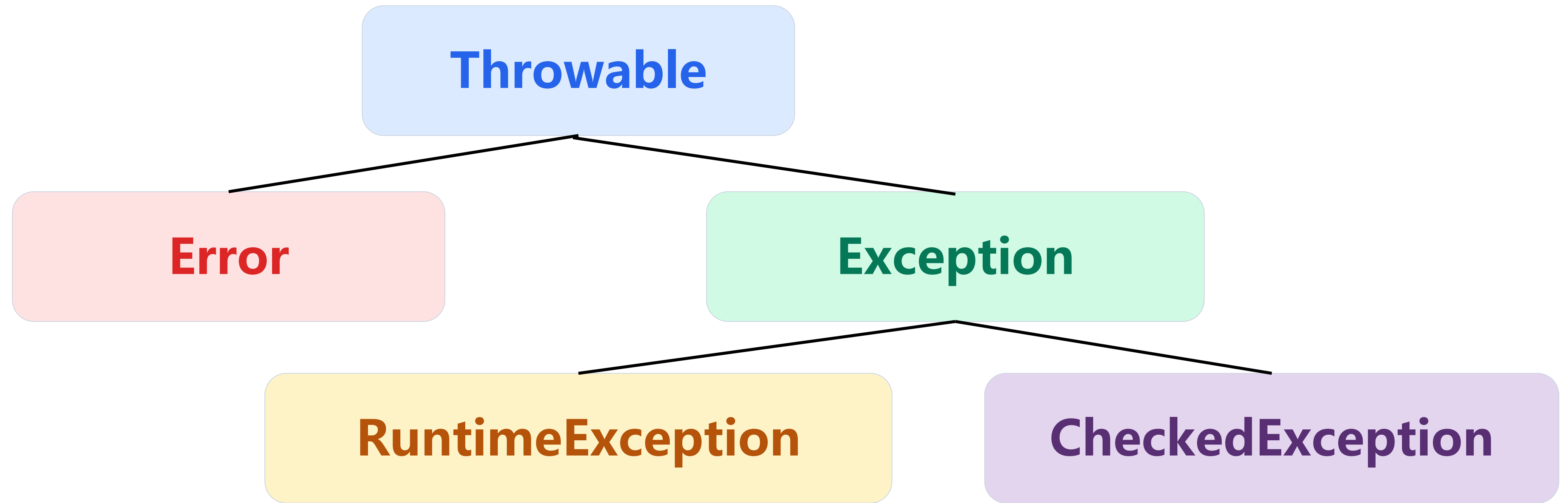
- 正常路径：方法顺利完成，返回正常结果
- 异常路径：方法无法完成任务，抛出一个异常对象
- 异常对象携带类型、消息、调用栈等信息
- 调用者可以捕获并处理，也可以继续向上传递

异常也是对象

```
if (score < 0 || score > 100) {  
    throw new IllegalArgumentException(  
        "分数必须在 0 到 100 之间"  
    );  
}
```

- 异常是某个异常类创建出来的对象
- 异常对象可以携带错误消息
- JVM 会记录异常发生时的调用栈
- 不同异常类型表示不同风险

异常继承树



- Error 通常表示严重问题，普通业务代码一般不处理
- Exception 是本节课重点，其中 checked exception 会被编译器强制检查

Checked vs Runtime Exception

Checked Exception: 编译器强制处理

- 外部环境风险更常见
- 文件不存在、数据库连接失败
- 必须 catch 或 throws
- 代表调用者应该认真面对的风险

Runtime Exception: 编译器不强制处理

- 代码逻辑问题更常见
- 空指针、越界、除零
- 可以不声明、不捕获
- 更应该通过修正逻辑避免

Checked 示例：文件读取失败

```
public List<Student> load(String path)
    throws IOException {
    return Files.readAllLines(Path.of(path))
        .stream()
        .map(Student::parse)
        .toList();
}
```

Runtime 示例：代码逻辑错误

```
Student s = null;  
System.out.println(s.name()); // NullPointerException
```

```
int average = total / count; // count 为 0 时  
                             // ArithmeticException
```

try/catch：把风险代码圈出来

```
try {  
    List<Student> students = loader.load("scores.csv");  
    service.save(students);  
    System.out.println("导入成功");  
} catch (IOException e) {  
    System.out.println("文件读取失败： " + e.getMessage());  
}
```

- try：放可能失败的代码
- catch：放失败之后的处理方案
- catch 的异常类型必须能接住 try 中的异常
- 处理异常不是“打印一下就结束”，而是要有恢复或报告策略

try/catch 的执行流程

- 异常发生后，try 中异常点之后的语句不会继续执行
- 多个 catch 时，从上往下找第一个匹配的 catch
- catch 执行结束后，如果没有继续抛异常，就回到后续普通代码

catch 要解决问题，而不只是吞掉问题

// 不推荐

```
try {  
    importScores();  
} catch (Exception e) {  
}
```

// 推荐

```
try {  
    importScores();  
} catch (IOException e) {  
    log.error("导入成绩文件失败", e);  
    ui.show("请检查文件路径");  
}
```

空 catch 会让错误信息消失。程序看起来没崩，但真正的问题更难定位。

finally: 无论成功失败, 都要收尾

```
FileInputStream in = null;  
try {  
    in = new FileInputStream("scores.csv");  
    // 读取文件  
} catch (IOException e) {  
    // 处理异常  
} finally {  
    if (in != null) {  
        in.close();  
    }  
}
```

- finally 常用于释放资源
- 比如关闭文件、连接、锁
- try 成功会执行 finally
- try 或 catch 抛异常, finally 也会执行

try-with-resources

```
try (BufferedReader br =  
    Files.newBufferedReader(Path.of("scores.csv"))) {  
    return br.lines()  
        .map(Student::parse)  
        .toList();  
}
```

- 资源写在 try 后面的括号里
- 结束时自动调用 close()
- 资源类必须实现 AutoCloseable
- 减少 finally 中手动关闭资源的错误

throws

```
public List<Student> loadScores(String path)
    throws IOException {
    return Files.readAllLines(Path.of(path))
        .stream()
        .map(Student::parse)
        .toList();
}
```

- throws 写在方法声明上
- 表示这个方法可能抛出某类异常
- 异常不在本方法处理，而是继续向上传
- throws 是方法 API 的一部分

throw

```
public static Student parse(String line) {  
    String[] parts = line.split(",");  
    int score = Integer.parseInt(parts[1]);  
  
    if (score < 0 || score > 100) {  
        throw new IllegalArgumentException("非法分数:  
" + score);  
    }  
    return new Student(parts[0], score);  
}
```

- throw 写在方法体内
- 后面跟一个异常对象
- 执行到 throw 时，异常一定发生
- 常用于前置条件检查和业务规则失败

throws vs. throw

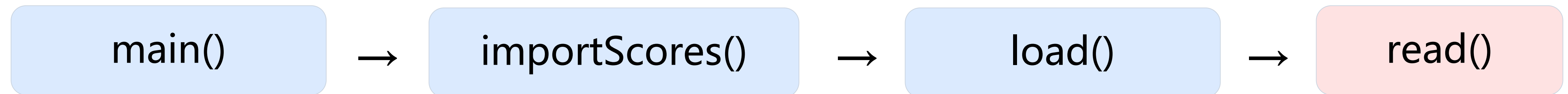
throw

- 写在方法体内
- 后面跟异常对象
- 一次只能抛一个对象
- 执行到这里一定抛出

throws

- 写在方法声明后
- 后面跟异常类型
- 可以声明多个异常类型
- 表示可能发生，由调用者处理

异常会沿调用栈向上传播



如果 read() 抛出 IOException, 异常会反方向向上传播, 直到遇到 catch。

某一层能处理, 就 catch
某一层处理不了, 就 throws
一直没人处理, 最终交给 JVM, 程序终止并打印调用栈

多个 catch：不同问题，不同处理

```
try {  
    importScores("scores.csv");  
} catch (NoSuchFileException e) {  
    ui.show("文件不存在，请重新选择");  
} catch (NumberFormatException e) {  
    ui.show("分数格式错误，请检查 CSV");  
} catch (IOException e) {  
    ui.show("读取失败，请稍后重试");  
}
```

- 越具体的异常越先 catch
- 同类问题可以放到同一个处理分支
- 最后可以用父类异常兜底
- 不要一开始就 catch Exception

catch 顺序：从小到大

// 正确

```
catch (NoSuchFileException e) { ... }  
catch (IOException e) { ... }  
catch (Exception e) { ... }
```

// 错误：后面的 IOException 永远到不了

```
catch (Exception e) { ... }  
catch (IOException e) { ... }
```

父类异常像更大的篮子。大篮子放前面，小篮子就永远接不到。

Java 7: 一个 catch 捕获多个异常

```
try {  
    importScores(path);  
} catch (IOException | SQLException e) {  
    log.error("导入失败", e);  
    ui.show("系统暂时不可用");  
}
```

- 多个异常处理方式相同时使用
- 异常类型之间用 | 分隔
- 不能出现重复类型
- 不能把父类和子类同时写在一个 multi-catch 中

自定义异常

```
class InsufficientBalanceException extends
Exception {
    public InsufficientBalanceException(
        int balance, int amount) {
        super("余额不足: 余额=" + balance
            + ", 需要=" + amount);
    }
}

void withdraw(int amount)
    throws InsufficientBalanceException {
    if (balance < amount) {
        throw new
InsufficientBalanceException(balance, amount);
    }
}
```

- 异常名直接表达业务含义
- 比单纯 RuntimeException 更清楚
- 异常消息应包含足够上下文
- 适合业务规则无法满足的场景

异常包装：底层错误，转换成业务语言

```
try {  
    Properties p = loadProperties("app.properties");  
    return AppConfig.from(p);  
} catch (IOException e) {  
    throw new ConfigLoadException(  
        "配置文件加载失败", e  
    );  
}
```

- 底层知道技术原因：IOException
- 上层更关心业务含义：配置加载失败
- 包装时保留 cause，方便调试
- 不要把原异常信息丢掉

异常声明是 API 的一部分

```
public interface ScoreImporter {  
    ImportResult importFrom(Path path)  
        throws IOException, InvalidScoreException;  
}
```

- 调用者会围绕异常声明写处理代码
- 随意增加 checked exception 会影响所有调用者
- 异常类型应该表达方法可能失败的方式
- API 越稳定，异常设计越要谨慎

重写方法时：不能扩大 checked exception

```
class Parent {  
    void save() throws IOException { }  
}
```

```
class Child extends Parent {  
    // 可以：不抛、抛 IOException、抛 IOException 的子类  
    void save() throws FileNotFoundException { }
```

```
    // 不可以：新增 SQLException  
    // void save() throws SQLException { }  
}
```

- 子类对象必须能替代父类对象
- 父类调用者只准备处理 IOException
- 子类不能突然增加新的 checked exception
- RuntimeException 不受这个规则限制

课堂判断题： 哪些重写合法？

```
class A {  
    void show() throws IOException, ClassNotFoundException,  
                ArithmeticException { }  
}
```

- A. void show() throws IOException, ClassNotFoundException, ArithmeticException, NullPointerException { }
- B. void show() throws IOException, ClassNotFoundException, ParseException { }
- C. void show() throws IOException, ClassNotFoundException { }
- D. void show() { }

答案： A、 C、 D。 B 错在新增 checked exception: ParseException。

finally 中不要 return

```
public static int f(int n) {  
    int r;  
    try {  
        r = n * n;  
        return r;  
    } finally {  
        r = 2;  
        return r;  
    }  
}
```

- finally 会在 return 前执行
- finally 自己 return 会覆盖 try/catch 的 return
- 也可能覆盖原本要抛出的异常
- finally 只适合做清理，不适合决定返回值

```
System.out.println(f(2)); // 输出 2
```

异常可能被覆盖

```
try {  
    readScores();    // 抛出  
    IOException  
} finally {  
    closeFile();    // 又抛出  
    CloseException  
}
```

- 清理阶段也可能失败
- 新的异常可能覆盖原始异常
- try-with-resources 会记录 suppressed exception
- 日志中要保留完整异常链

调试异常

- 异常类型：例如 NullPointerException、IOException、SQLException
- 异常消息：它通常说明哪个值、哪个文件、哪个操作失败
- 调用栈第一处自己的代码：那里通常是最该检查的位置
- 不要只看最后一行，也不要被很长的框架调用栈吓到

异常使用建议

- 不要用异常代替简单判断：空栈先 isEmpty, 再 pop
- 不要 catch 后什么都不做
- 不要所有错误都抛 RuntimeException
- 发现非法状态时尽早抛出，让错误靠近原因
- 在真正知道怎么恢复或提示用户的地方捕获异常
- 记录异常时保留原始 cause

成绩导入器应该怎么处理？

- scores.csv 不存在：应该 catch 还是 throws？在哪里提示用户？
- 某一行分数是 abc：应该抛什么异常？异常消息要包含什么？
- 数据库保存失败：底层 SQLException 是否要包装成业务异常？
- 文件流需要关闭：手写 finally 还是 try-with-resources？

总结

- 异常把“不能正常完成任务”的情况从正常返回值中分离出来
- checked exception 必须处理或声明, runtime exception 不强制
- try/catch/finally 控制异常发生后的执行流程
- throw 是抛出异常对象, throws 是声明方法风险
- 异常设计影响 API, 可维护性和调试效率

异常不是为了让程序报错, 而是为了让程序把意外说清楚。